

## EDA DATA CLEANING

### AIM

To perform exploratory data analysis on the E-Commerce Customer Insights and Churn Dataset by examining the dataset structure, dimensions, data types, statistical characteristics, missing values, country-wise and category-wise purchase distributions, product-wise purchase patterns, temporal purchase trends, purchase amount analysis, interactive filters, parameter-based analysis, and creating an interactive dashboard

### ALGORITHM

1. Import the required libraries such as Pandas, NumPy, Matplotlib, and Seaborn.
2. Load the E-Commerce Customer Insights and Churn Dataset into a Pandas DataFrame.
3. Display the first five and last five records of the dataset.
4. Display the dataset information, shape, column names, and data types.
5. Generate descriptive statistical information for numerical attributes.
6. Identify the number of missing values in each column.
7. Remove blank rows and duplicate records from the dataset.
8. Perform data cleaning and standardization on the required categorical and numerical attributes.
9. Calculate the Total Purchase Amount using unit\_price and quantity.
10. Analyze the distribution of purchases across different countries and categories.
11. Analyze the distribution of purchases across different products.
12. Analyze the purchase trend over time using the order\_date attribute.
13. Calculate and analyze the Purchase Amount Percentage for the different categories or countries.
14. Create interactive visualizations including bar charts, pie charts, line charts, Country Filter, Minimum Purchase Amount Parameter, and an interactive Retail Sales Analysis Dashboard.
15. Export the cleaned dataset into CSV format and display a confirmation message after successful file creation.

### CODE

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

print("First 5 Records")
print(df.head())

print("\nDataset Information")
df.info()

print("\nMissing Values")
print(df.isnull().sum())

print("\nStatistical Summary")
print(df.describe())

print("\nDataset Shape:", df.shape)
print("Column Names:", df.columns.tolist())
date_columns = ["signup_date", "last_purchase_date", "order_date"]

for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors="coerce")
df["signup_year"] = df["signup_date"].dt.year
df["signup_month"] = df["signup_date"].dt.month
df["last_purchase_year"] = df["last_purchase_date"].dt.year
df["last_purchase_month"] = df["last_purchase_date"].dt.month
df["order_year"] = df["order_date"].dt.year
df["order_month"] = df["order_date"].dt.month
df["order_day"] = df["order_date"].dt.day
```

```

df["order_weekday"] = df["order_date"].dt.weekday

df["customer_lifetime_days"] = (
    df["last_purchase_date"] - df["signup_date"]
).dt.days

df.drop(date_columns, axis=1, inplace=True)
df["Churn"] = (
    df["subscription_status"].str.lower().str.strip() == "cancelled"
).astype(int)

print("\nChurn Distribution")
print(df["Churn"].value_counts())
df.drop("subscription_status", axis=1, inplace=True)
df.drop(["order_id", "customer_id", "product_id"], axis=1, inplace=True)
for col in df.columns:
    if col != "Churn":
        if df[col].dtype == "object":
            df[col] = df[col].fillna(df[col].mode()[0])
        else:
            df[col] = df[col].fillna(df[col].median())
encoder = LabelEncoder()

for col in df.select_dtypes(include="object").columns:
    df[col] = encoder.fit_transform(df[col].astype(str))
X = df.drop("Churn", axis=1)
y = df["Churn"]

print("\nNumber of Features:", X.shape[1])
print("Number of Records:", X.shape[0])

```

```

X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print("Training Records:", X_train.shape[0])
print("Testing Records:", X_test.shape[0])

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nCRISP-DM Accuracy:", round(accuracy * 100, 2), "%")

```

## OUTPUT

```

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   order_id              2000 non-null   object
1   customer_id           2000 non-null   object
2   age                   2000 non-null   int64
3   product_id            2000 non-null   object
4   country                2000 non-null   object
5   signup_date            2000 non-null   object
6   last_purchase_date     2000 non-null   object
7   cancellations_count    2000 non-null   int64
8   subscription_status    2000 non-null   object
9   order_date             2000 non-null   object
10  unit_price             2000 non-null   float64
11  quantity               2000 non-null   int64
12  purchase_frequency     2000 non-null   int64
13  preferred_category     2000 non-null   object
14  product_name           2000 non-null   object
15  category               2000 non-null   object
16  gender                 2000 non-null   object
dtypes: float64(1), int64(4), object(12)
memory usage: 265.8+ KB

Missing Values
order_id      0
customer_id   0
age           0
product id    0

```

# Statistical Summary

|       | age         | cancellations_count | unit_price  | quantity    | \ |
|-------|-------------|---------------------|-------------|-------------|---|
| count | 2000.000000 | 2000.000000         | 2000.000000 | 2000.000000 |   |
| mean  | 44.080500   | 2.441500            | 324.719460  | 4.948500    |   |
| std   | 14.798091   | 1.702359            | 361.548405  | 2.557757    |   |
| min   | 18.000000   | 0.000000            | 2.850000    | 1.000000    |   |
| 25%   | 31.000000   | 1.000000            | 111.127500  | 3.000000    |   |
| 50%   | 45.000000   | 2.000000            | 206.515000  | 5.000000    |   |
| 75%   | 57.000000   | 4.000000            | 384.507500  | 7.000000    |   |
| max   | 69.000000   | 5.000000            | 1991.630000 | 9.000000    |   |

customer

|        | order_id | customer_id | age         | product_id | country | signup_date | \ |
|--------|----------|-------------|-------------|------------|---------|-------------|---|
| count  | 2000     | 2000        | 2000.000000 | 2000       | 2000    | 2000        |   |
| unique | 2000     | 2000        | NaN         | 2000       | 6       | 926         |   |
| top    | ORD6983  | CUST2983    | NaN         | PROD2183   | Germany | 04-10-2023  |   |
| freq   | 1        | 1           | NaN         | 1          | 360     | 8           |   |
| mean   | NaN      | NaN         | 44.080500   | NaN        | NaN     | NaN         |   |
| std    | NaN      | NaN         | 14.798091   | NaN        | NaN     | NaN         |   |
| min    | NaN      | NaN         | 18.000000   | NaN        | NaN     | NaN         |   |
| 25%    | NaN      | NaN         | 31.000000   | NaN        | NaN     | NaN         |   |
| 50%    | NaN      | NaN         | 45.000000   | NaN        | NaN     | NaN         |   |
| 75%    | NaN      | NaN         | 57.000000   | NaN        | NaN     | NaN         |   |
| max    | NaN      | NaN         | 69.000000   | NaN        | NaN     | NaN         |   |

|        | last_purchase_date | cancellations_count | subscription_status | order_date | \ |
|--------|--------------------|---------------------|---------------------|------------|---|
| count  | 2000               | 2000.000000         | 2000                | 2000       |   |
| unique | 1078               | NaN                 | 3                   | 1106       |   |
| top    | 11/30/2023         | NaN                 | active              | 4/24/2024  |   |
| freq   | 7                  | NaN                 | 1204                | 7          |   |
| mean   | NaN                | 2.441500            | NaN                 | NaN        |   |
| std    | NaN                | 1.702359            | NaN                 | NaN        |   |
| min    | NaN                | 0.000000            | NaN                 | NaN        |   |
| 25%    | NaN                | 1.000000            | NaN                 | NaN        |   |
| 50%    | NaN                | 2.000000            | NaN                 | NaN        |   |
| 75%    | NaN                | 4.000000            | NaN                 | NaN        |   |
| max    | NaN                | 5.000000            | NaN                 | NaN        |   |

|        | unit_price  | quantity    | purchase_frequency | preferred_category | \ |
|--------|-------------|-------------|--------------------|--------------------|---|
| count  | 2000.000000 | 2000.000000 | 2000.000000        | 2000               |   |
| unique | NaN         | NaN         | NaN                | 5                  |   |
| top    | NaN         | NaN         | NaN                | Clothing           |   |
| freq   | NaN         | NaN         | NaN                | 419                |   |
| mean   | 324.719460  | 4.948500    | 25.158000          | NaN                |   |
| std    | 361.548405  | 2.557757    | 14.158752          | NaN                |   |
| min    | 2.850000    | 1.000000    | 1.000000           | NaN                |   |
| 25%    | 111.127500  | 3.000000    | 13.000000          | NaN                |   |
| 50%    | 206.515000  | 5.000000    | 25.000000          | NaN                |   |
| 75%    | 384.507500  | 7.000000    | 37.000000          | NaN                |   |
| max    | 1991.630000 | 9.000000    | 49.000000          | NaN                |   |

## RESULT